

TimeTracker

AI-Driven Software Development — Project Report

Ali Akbar Baloch

Summer Semester 2026

Contents

1	System Architecture and Design Decisions	2
1.1	High-level architecture	2
1.2	Authentication design	2
1.3	Database and persistence	2
1.4	Backend layer structure	2
1.5	Project sharing data model	3
1.6	Task-project many-to-many relationship	3
1.7	Accumulated timer design	3
1.8	Frontend state management	3
1.9	UI design system	3
1.10	Performance design	4
2	Used LLMs and AI-based Tools	4
2.1	How Claude Code was used	4
3	Where Did the AI Tools Work Well?	5
3.1	Consistent boilerplate generation	5
3.2	Comprehensive test authoring	5
3.3	Complex cross-cutting features	5
3.4	CI failure diagnosis	5
3.5	UI/UX overhaul	6
4	Where Did the AI Tools Not Work as Intended?	6
4.1	Playwright E2E test flakiness	6
4.2	Incremental bugs introduced after merge	7
4.3	Overly broad initial implementations	7
5	Custom Features	8
5.1	US-026 — Project Time Budgets	8
5.2	US-027 — Task Templates	8
5.3	US-028 — Productivity Analytics	9

1 System Architecture and Design Decisions

1.1 High-level architecture

TimeTracker is a full-stack single-page application structured around a clear separation of concerns: a **Spring Boot 3 / Java 21 backend** that exposes a RESTful JSON API, a **React 19 + Vite frontend** that communicates with it exclusively over HTTP, and a **file-based H2 database** for persistence. The backend uses Spring Data JPA for the persistence layer, Spring Security 6 for authentication, and jjwt 0.12.6 for token generation and validation.

1.2 Authentication design

Authentication is **stateless JWT-based**: on login the backend signs a JWT with an HMAC-SHA256 key and returns it to the client; the client stores it in `localStorage` and attaches it as a **Bearer** header on every subsequent request. There are no server-side sessions. This design was chosen because it integrates cleanly with single-page applications (no cookie/CSRF complexity), scales horizontally without shared session state, and keeps the backend fully stateless.

A more sophisticated authentication mechanism, such as OAuth2 or OpenID Connect with an external identity provider, could have been integrated. However, the custom JWT approach was deliberately chosen to keep the application self-contained with zero external dependencies and to give full control over token issuance, expiry, and revocation.

BCrypt at cost factor 10 hashes all stored passwords; the random salt ensures that two identical passwords produce different hashes, neutralising pre-computation attacks.

1.3 Database and persistence

H2 is used in two modes: **file-based** (`jdbc:h2:file:./data/timetracker`) in development and Docker Compose, and **in-memory** (`jdbc:h2:mem:testdb`) for the test profile. This decision eliminated every external dependency: a developer can clone the repository, run `docker compose up --build`, and have a fully functional stack within minutes without installing or configuring a separate database server.

The DDL strategy is **create-drop** in tests and **update** in development, so the schema is auto-created on first boot and survives restarts without manual SQL steps.

1.4 Backend layer structure

The backend follows a strict four-layer convention:

- **entity/** — JPA entities (`User`, `Task`, `Project`, `ProjectMember`, `TaskTemplate`).
- **repository/** — Spring Data JPA interfaces; method names derive queries; `@Query` used only when derivation is insufficient.
- **service/** — all business logic, the exclusive mutation testing (PITest) target.
- **controller/** — thin REST layer; delegates immediately to the service, enforces HTTP-level concerns only.

In contrast to the initial design, which planned a single `ProjectController` for all project operations, the implementation split project concerns across `ProjectController` (CRUD), `ProjectSummaryController` (time aggregation), and `ProjectMemberController`-adjacent endpoints within the same controller class. This kept each method short and made the ownership/membership access checks easier to reason about in isolation.

1.5 Project sharing data model

Sharing is implemented through a dedicated `project_members` join table with columns (`project_id`, `user_id`, `role`, `joined_at`) and a unique constraint on (`project_id`, `user_id`). The `project.user_id` column was retained as the original owner foreign key for backward compatibility. All read operations use a `findByIdAndMember` query (any member may read); write operations use `findByIdAndUser` (owner only). A `MembershipSeeder` component back-fills `OWNER` rows on application startup for all projects created before the sharing feature was added.

1.6 Task-project many-to-many relationship

Tasks and projects are connected through a `task_projects` join table managed with `@ManyToMany`. This allows a single task to belong to multiple projects simultaneously. Cascading is managed manually: the join table is cleared before any task or project delete to prevent foreign-key constraint violations.

1.7 Accumulated timer design

When a user starts a task template for the second or third time, a new `Task` row is created (preserving a complete audit trail) but the timer is initialised to the sum of all previous completed sessions. This is modelled via a `totalPreviousSeconds` column (bigint, default 0) on the `Task` entity. The frontend timer displays `totalPreviousSeconds + (now - startTime)`, giving the user a continuously growing counter across template restarts without changing the underlying entity semantics.

1.8 Frontend state management

The frontend intentionally avoids Redux or any global state library. State is managed through two React Contexts:

- **AuthContext** — JWT token, user email, display name, and timezone; persisted in `localStorage`.
- **TimerContext** — the active running task and a live elapsed string, rehydrated from `GET /api/tasks/active` on every mount (no browser storage).

A **ThemeContext** was added in the UI/UX overhaul ([PR #130](#)) to support light, dark, and system-preference modes via CSS custom properties on the `<html>` element.

1.9 UI design system

In contrast to the initial design, which used inline styles and scattered CSS classes, the final frontend uses a **CSS custom-property token system** (`App.css`) with named semantic

tokens for every colour, surface, shadow, border radius, and spacing value. All theme variants (light and dark) are expressed as overrides on the `[data-theme]` attribute, requiring no JavaScript for colour switching. A fixed sidebar at 224 px replaces the original horizontal topbar, improving usability on wide monitors and enabling persistent navigation without page jumps.

1.10 Performance design

Database indexes on `tasks(user_id, start_time)`, `tasks(user_id, end_time)`, and `projects(user_id, parent_project_id)` cover the hot query paths. All task-fetching queries use `LEFT JOIN FETCH t.projects` to eliminate the N+1 query problem. The Dashboard summary is computed in a single `GET /api/dashboard/summary` call that returns today/week totals, the running task, and the top-5 projects in one response, eliminating any waterfall.

2 Used LLMs and AI-based Tools

The entire project was developed using a single AI tool: **Claude Code** (Anthropic's CLI agent) running **Claude Sonnet 4.6**, accessed through the VS Code extension. No other LLM or AI-based tool was used.

2.1 How Claude Code was used

Claude Code was used as an **interactive pair-programming agent** for every phase of development:

- **Architecture and design** — discussing and deciding on the layered backend structure, JWT statelessness, H2 file-persistence, and the many-to-many task-project model before writing a single line of code.
- **Feature implementation** — writing JPA entities, Spring service and controller classes, React page components, CSS, and Axios API wrappers for all 28 user stories and 4 non-functional requirements.
- **Test authoring** — generating the full suite of 457 JUnit/Mockito backend tests and 326 Vitest/React Testing Library frontend tests, covering unit, integration, and component levels.
- **E2E test authoring** — writing all 28 Playwright spec files (one per user story/NFR), including the shared `auth.ts` helper that registers users via API and injects JWT directly into `localStorage` to bypass the login form.
- **CI pipeline design and debugging** — setting up the GitHub Actions pipeline (backend, frontend, and Playwright system-test jobs in Docker), diagnosing CI failures, and iterating on fixes.
- **Bug investigation and fixing** — all fix PRs (e.g. analytics year sync, accumulated timer, NFR-003 layout overflow, Playwright strict-mode violations) were diagnosed and resolved through a conversation with the agent.
- **Documentation** — writing issue descriptions, PR summaries, the README (includ-

ing the full test table, API reference, and design-decisions section), and this report.

The `CLAUDE.md` file at the repository root served as a standing briefing document for the agent. It encoded process rules (one issue per PR, CI must never fail, commit style), the full architecture description, and a growing list of design decisions so that every new conversation started with full project context without needing re-explanation.

3 Where Did the AI Tools Work Well?

3.1 Consistent boilerplate generation

The most immediate win was the elimination of boilerplate. Each user story followed the same pattern: entity $\rightarrow$ repository $\rightarrow$ service $\rightarrow$ controller $\rightarrow$ DTO $\rightarrow$ frontend page. Claude Code understood this convention from `CLAUDE.md` and generated all layers coherently in a single turn, including field validation, error handling, and the correct Spring Security integration. What would have taken hours of repetitive typing per story was reduced to minutes.

3.2 Comprehensive test authoring

Claude Code produced test suites that went well beyond happy-path coverage. For example, `TaskServiceTest` (41 tests, [PR #35](#) and later expansions) uses `ArgumentCaptor` to verify the exact fields written to the database on every save call, specifically to kill setter-mutation `PITest` mutants. The agent understood that `PITest` kills mutations by detecting when a field is set to the wrong value, and wrote tests that specifically validated each field assignment — a non-trivial testing strategy. The final suite achieves $\geq 90\%$ JaCoCo line coverage, $\geq 80\%$ `PITest` mutation-test strength, and $\geq 90\%$ Vitest statement coverage across all three metric categories.

3.3 Complex cross-cutting features

The project-sharing feature ([Issue #59](#), [PR #66](#)) required coordinating changes across the entity model, four repository methods, two service classes, access-control guards in every controller, the frontend project list, project detail page, and the invitation form. Claude Code designed the full `project_members` table approach, wrote the `MembershipSeeder` to back-fill ownership rows for pre-existing data, and produced the 17-test `ProjectSharingTest` integration suite in one coherent turn. The breadth of that change (touching 12+ files) would have been highly error-prone if implemented manually.

3.4 CI failure diagnosis

When the CI pipeline failed, Claude Code diagnosed root causes from raw error output with high accuracy. The most precise example was the CI workspace-mount bug ([PR #58](#)): the GitHub Actions backend job mounted only `backend/` as `/workspace`, so `LocalDeployabilityTest`'s `../docker-compose.yml` path resolved to the container filesystem root. Claude Code identified the exact volume-mount line in the YAML, explained the resolution order, and produced the one-line fix immediately.

3.5 UI/UX overhaul

The UI/UX redesign ([Issue #129](#), [PR #130](#)) produced a complete CSS custom-property token system, a light/dark/system theme switcher, skeleton shimmer loaders, a toast notification system, a fixed sidebar with mobile hamburger menu, route transition animations, and an illustrated SVG empty state — all in a single large PR. The visual result was production-quality, not a rough draft, and required no manual CSS iteration.

4 Where Did the AI Tools Not Work as Intended?

4.1 Playwright E2E test flakiness

Writing Playwright end-to-end tests proved to be the most challenging area. The first full E2E run on CI failed with 30 test failures across 14 spec files ([PR #116](#)). The root causes were varied and subtle:

- **Strict-mode violations:** Playwright’s strict mode throws when a locator matches more than one element. A success toast (`role="alert"`) was still in the DOM when a subsequent error `<p role="alert">` appeared, causing `getByRole('alert')` to resolve to two elements. The fix required adding `.filter({ hasText: /pattern/i })` ([PR #132](#)).
- **Timing races:** `waitForResponse` fires at the network layer before the browser’s JavaScript `fetch().then()` callback runs and React re-renders. Reading `rows.count()` immediately after the event returned the pre-update DOM count. The fix was to use `expect(rows).not.toHaveCount(oldCount, { timeout: 8_000 })` to poll until the DOM reflected the new state.
- **Time-of-day-dependent failures:** Three `TaskControllerListFilterTest` tests used `Instant.now().truncatedTo(DAYS)` as a window boundary. Tasks created about an hour before the test ran appeared outside the window during the first hour of a UTC day. The fix replaced the absolute day boundary with a relative 2-hour window.
- **DOM detach after React re-render:** Stopping a timer triggered `setActiveTask(null)` followed by `setElapsed('00:00:00')` in rapid succession, briefly unmounting the start button. Playwright’s `click()` kept retrying until the 30-second timeout. The fix was to stop the timer via the API in `beforeEach` before navigating to the page.

The first-generation tests were generated without a full mental model of the browser event loop and React’s asynchronous reconciliation. Three follow-up PRs ([#117](#), [#118](#), [#132](#)) were needed before the full suite passed reliably.

Adaptation: providing the agent with the exact CI log output (including which locator failed, what elements were found, and the stack trace) on each iteration was the key to getting accurate fixes. Vague prompts like “fix the failing tests” produced generic solutions; specific prompts like “here is the exact Playwright error, here is the component code, explain the timing issue” produced targeted and correct fixes.

4.2 Incremental bugs introduced after merge

Several features were implemented correctly at a high level but contained subtle logic errors that only manifested later:

- **Analytics year boundary bug** (Issue #125, PR #124, PR #127): the weekly-pattern service used `year >= currentYear` to decide the upper time bound, which caused future years (e.g. 2027) to display current-year data. The correct condition is `to = min(now, yearEnd)`, with `from = yearStart` for future years forcing `from > to` so the query returns zero rows. This required two separate PRs to fully fix — the first corrected the frontend year propagation, the second fixed the backend boundary logic.
- **Accumulated timer omission** (Issue #126, PR #128): the initial Task Templates implementation created a new task on every template start but always initialised the timer to zero, ignoring previous sessions. The `totalPreviousSeconds` column and the accumulation logic were added in a subsequent PR after the templates feature was already merged and in use.
- **NFR-003 layout overflow** (PR #132): the `.page-content` CSS class contained `margin-left: var(--sidebar-w)`, and the outer wrapper div in `Layout.jsx` applied the same offset via an inline style. The doubled offset (448 px instead of 224 px) pushed the dashboard content well beyond the 1024 px viewport, causing `body.scrollWidth` to reach 1179 px. The CSS grid additionally expanded its `1fr` column to its min-content size rather than its available track size because `1fr` defaults to `minmax(auto, 1fr)`. Two separate commits were needed: removing the duplicate margin, then changing `1fr` to `minmax(0, 1fr)` with `min-width: 0` on grid children.

Adaptation: in each case, the agent fixed the issue correctly once given the CI failure output and the specific file. The lesson learned was to run the E2E test suite locally and inspect NFR checks (layout at 1024 px) before pushing, rather than relying on CI as the first line of detection. Extending the CLAUDE.md “Key Design Decisions” section with the root cause and fix pattern for each class of bug prevented the same mistake from recurring in subsequent PRs.

4.3 Overly broad initial implementations

For several features, the agent produced working code that was more complex than needed. The analytics service originally implemented three separate repository query methods and a custom aggregation loop. The simpler final design reuses the existing `findByUserAndStartTimeBetween` method (already used by the daily/weekly overview) and performs all grouping in Java streams, which meant zero new repository methods and no new database indexes were required. Getting to this simpler design required explicitly asking the agent “can this use the existing repository method instead of a new one?” — the first-pass answer was always to introduce new infrastructure.

5 Custom Features

Three custom features were designed and implemented beyond the mandatory project requirements.

5.1 US-026 — Project Time Budgets

[Issue #63](#) · [PR #70](#)

A project can optionally carry a `budgetHours` value (a positive decimal, e.g. `40.0`). When set, the project detail page, project list, and dashboard top-projects list all display a colour-coded progress bar: green below 80% usage (`ON_TRACK`), orange at 80–99% (`WARNING`), and red at or above 100% (`OVER_BUDGET`).

Key design choices:

- The `budgetHours` column is nullable `Double` on the `Project` entity. A null value means no budget is enforced and no progress bar is rendered.
- Budget usage is an **all-time aggregate** across the full project subtree and all members, intentionally independent of the date-range filter on the project summary.
- `ProjectService.computeBudgetStatus` is a `static` helper so that both `ProjectService.getProjectSummary` and `DashboardService` can compute budget status without a circular dependency between the two services.

5.2 US-027 — Task Templates

[Issue #64](#) · [PR #71](#) · [PR #128](#)

A `TaskTemplate` entity stores a user-defined name, an optional description, and optional project associations. Clicking **Start** on a template creates a new running task pre-populated with the template’s description and project links. Starting the same template multiple times accumulates time across sessions: the timer begins from the sum of all previous completed sessions for that description rather than from zero.

Key design choices:

- Templates are **strictly private**; ownership is enforced via `findByIdAndUser`, returning 404 (not 403) to avoid leaking template existence to other users.
- The `startFromTemplate` method delegates to the existing `TaskService.startTask(userEmail, request, projectIds)` overload rather than duplicating the 409-if-running check.
- Project associations on a template are resolved with `findByIdAndMember`, preventing users from associating arbitrary project IDs with their templates.
- The accumulated-timer design keeps each session as a **separate Task row** (complete audit trail) while showing the user a continuously growing total via `totalPreviousSeconds + (now - startTime)`.

5.3 US-028 — Productivity Analytics

[Issue #65](#) · [PR #72](#) · [Issue #131](#) · [PR #132](#)

A dedicated Analytics page offers three visualisations:

1. **Activity Heatmap** — a GitHub-style 52–53-week grid (7 rows $\times$ N columns, Monday-first). Each cell represents one calendar day colour-coded in four levels based on its share of the year’s maximum tracked time. Day boundaries are computed in the user’s preferred timezone using `ZonedDateTime`. A year selector (current year ± 10) lets users browse historical data.
2. **Average Hours by Day of Week** — a bar chart showing average tracked seconds per weekday (Mon–Sun) over the last 12 weeks of the selected year. The average is `total / weeks` (not `total / days-with-data`), giving a true per-week mean.
3. **Shared Project Breakdown** — for each shared project where at least two members tracked time in the window, a ranked list of contributor bars showing each person’s share as a percentage. Projects with only one contributor or zero total seconds are excluded.

Key design choices:

- All three endpoints reuse the existing `findByUserAndStartTimeBetweenOrderByStartTimeAsc` repository method and the `(user_id, start_time)` index — no new database indexes were required.
- Future years return zero results by design: `to = min(now, yearEnd)` means `from = yearStart > to`, so the `BETWEEN` clause returns no rows.
- The frontend heatmap cell array is built from a flat list padded to complete weeks using `(Jan1.getDay() + 6) % 7` (converts JavaScript’s Sunday=0 to Monday=0), rendered via `grid-auto-flow: column` so week columns emerge naturally from a row-major array.

References